

WS63 星闪(SLE) 三节点 车载信息显示系统

GEC6818 上位机完整讲解

项目名称	SLE CID 车载信息显示系统
硬件平台	HiSilicon WS63E × 3 块, GEC6818 × 1 块
无线协议	星闪 SLE (Spark-Link Essence)
上位机 OS	Linux (ARM Cortex-A53, 800×480 触摸屏)
云平台	OneNET 物联网平台 (MQTT 3.1.1)
天气 API	ShowAPI 合肥实时天气
编译工具链	arm-linux-gcc (交叉编译)

第一章 项目总览与系统架构

1.1 项目目标

本项目以 HiSilicon WS63E 星闪模组为核心，构建一套包含三个 WS63 节点 和一块 GEC6818 ARM 上位机的车载信息显示（CID）系统。系统功能覆盖：

- 传感器采集：Board 1 读取 SHT30 温湿度传感器和 MQ-135 空气质量传感器；
- 无线传输：三块 WS63 通过星闪（SLE）协议互联，Board 1 作 Server（主节点），Board 2 和 Board 3 作 Client（从节点）；
- 有线上报：Board 1 经 UART 串口将传感器帧发送给 GEC6818；
- CID 显示：GEC6818 在 800×480 触摸屏上显示车载仪表盘界面（BMP 背景 + 文字叠加）；
- 触摸交互：点击 8 个分区触发时间/天气/温湿度/空气质量查看、座椅电机上下调节、氛围灯开关；
- 云端上传：GEC6818 通过 MQTT 将传感器数据上传至 OneNET 物联网平台；
- 天气查询：GEC6818 通过 HTTP Socket 调用 ShowAPI 获取合肥实时天气。

1.2 硬件节点角色

节点	型号	角色	主要职责
Board 1	WS63E	SLE Server（主节点）	采集 SHT30 温湿度 + MQ-135 空气质量；经 UART 串口上报传感器帧给 GEC6818；接收 GEC6818 发来的控制指令，通过 SLE Notify 分别转发给 Board 2（电机）和 Board 3（灯光）
Board 2	WS63E	SLE Client（座椅节点）	接收 Board 1 的 SLE Notify 帧；驱动座椅步进电机正/反转（指令 F5/R5）
Board 3	WS63E	SLE Client（灯光节点）	接收 Board 1 的 SLE Notify 帧；控制氛围灯 LED 开/关（指令 L1/L0）
GEC6818	ARM Linux	上位机（UI 主机）	接收 UART 传感器帧；渲染 CID 界面（LCD Framebuffer）；处理触摸事件；拉取天气；上传 OneNET MQTT

1.3 整体数据流

环节	发送方	接收方	协议/接口	内容
1	SHT30/MQ-135	Board 1 MCU	I2C / ADC	温湿度、空气质量原始数据

环节	发送方	接收方	协议/接口	内容
2	Board 1 UART TX	GEC6818 UART1	UART 115200 8N1	11字节传感器帧 (AA 55 ... 0D)
3	GEC6818	Board 1 UART RX	UART 115200 8N1	控制指令: F5/R5 (电机) 、 L1/L0 (灯光)
4	Board 1	Board 2	SLE Notify	座椅电机步进指令 F5/R5
5	Board 1	Board 3	SLE Notify	氛围灯指令 L0/L1
6	GEC6818 主线程	LCD Framebuffer	/dev/fb0 mmap	BMP背景 + 文字叠加渲染
7	GEC6818	ShowAPI 服务器	HTTP GET Socket	合肥实时天气 JSON
8	GEC6818	OneNET MQTT Broker	MQTT 3.1.1 QoS0	温度/湿度/空气质量 JSON

第二章 星闪（SLE）协议基础

2.1 什么是星闪

星闪（SparkLink）是由华为主导的面向近场通信的新一代无线连接标准，分为两个子集：SLE（SparkLink Essence，基础版）和 SLB（SparkLink Basic）。SLE 工作在 2.4 GHz 频段，支持 GFSK/极化码等多种调制编码方式，最高物理层速率可达 12 Mbps（MCS10，4 MHz 带宽），时延可低至毫秒级。

与蓝牙 BLE 相比，SLE 具有更低的接入时延、更强的抗干扰能力和更好的组网灵活性，是华为 HarmonyOS 生态下取代 BLE 的核心无线技术。

提示：本项目使用的 SLE 协议栈由 HiSilicon SDK 提供，开发者只需调用 API，无需关心底层 RF 驱动实现。SDK 路径为 fbb_ws63/src/。

2.2 SLE SSAP 服务层

SLE 的上层服务采用 SSAP（SparkLink Service Attribute Protocol），类似 BLE 的 GATT。其核心概念如下：

概念	对应 BLE 概念	说明
Server	GATT Server	提供数据/服务的一侧（本项目 Board 1）
Client	GATT Client	主动连接并读写数据的一侧（本项目 Board 2）
Service	Service	一组相关属性的集合，用 UUID 标识
Property	Characteristic	服务下的数据单元，有 Read/Write/Notify 权限
Descriptor	Descriptor	Property 的附加描述，本项目用于使能 Notify
Announce	Advertisement	Server 广播自身存在，Client 扫描后发起连接
Notify/Indicate	Notify/Indicate	Server 主动向 Client 推送数据

2.3 关键 API 速查

API 函数	用途
enable_sle()	使能 SLE 协议栈
ssaps_register_server()	注册 SSAP Server，获得 server_id
ssaps_add_service_sync()	同步添加 Service，返回 service_handle

API 函数	用途
ssaps_add_property_sync()	同步添加 Property (特征值), 返回 property_handle
ssaps_add_descriptor_sync()	为 Property 添加 Descriptor (如使能 Notify)
ssaps_start_service()	启动 Service, 使其对客户端可见
ssaps_notify_indicate()	Server 主动 Notify 数据给已连接的 Client
ssaps_register_callbacks()	注册 SSAP 回调 (读/写/MTU变更/服务启动)
sle_announce_seek_register_callbacks()	注册广播/扫描回调
sle_set_announce_param()	设置广播参数 (间隔、TX Power 等)
sle_set_announce_data()	设置广播数据与扫描响应数据
sle_start_announce()	开始广播 (Server 调用)
sle_start_scan()	开始扫描 (Client 调用)
sle_connect_remote_device()	向目标地址发起连接请求
sle_connection_register_callbacks()	注册连接状态回调
sle_update_connect_param()	更新连接参数 (连接间隔/超时)
sle_set_data_len()	设置空中包长度 (影响吞吐率)
sle_set_phy_param()	设置 PHY 参数 (调制方式、速率等)

注意: Client 向 Server 写入时, 必须使用 Property 的 handle (property_handle), 而不是 Descriptor 的 handle, 否则 Server 端的 ssaps_write_request_cb 回调不会被触发。这是初学者最常踩的坑。

第三章 WS63 Board 1 —— SLE Server 传感器节点

3.1 Board 1 的职责

- 读取 SHT30 温湿度传感器 (I2C) 和 MQ-135 空气质量传感器 (ADC) ；
- 将传感器数据打包成 11 字节自定义帧，经 UART (/dev/ttySAC1) 发送给 GEC6818；
- 注册为 SLE Server，持续广播，等待 Board 2 (电机节点) 和 Board 3 (灯光节点) 连接；
- 同时维护两个 SLE 连接 (conn_id)，管理双 Client 的 Notify 路由；
- 接收 GEC6818 经 UART 发来的控制指令 (F5/R5/L1/L0)，按指令类型分别通过 SLE Notify 转发给 Board 2 (电机) 或 Board 3 (灯光) 。

3.2 SLE Server 初始化流程

Server 初始化由 sle_speed_server_init() 完成，调用顺序如下：

步骤	函数调用	说明
1	uapi_watchdog_disable()	关闭看门狗，防止初始化过长被复位
2	enable_sle()	使能 SLE 协议栈，触发 sle_enable_cbk
3	sle_speed_server_set_nv()	配置 NV 项：设置 BTC 发射功率档位为 7
4	sle_conn_register_cbks()	注册连接状态回调 (connect_state_changed_cb 等)
5	sle_ssaps_register_cbks()	注册 SSAP 读/写/MTU/启动回调
6	sle_uuid_server_add()	注册 Server → 添加 Service → 添加 Property → 启动 Service
7	sle_uuid_server_adv_init()	设置广播参数、广播数据，开始广播
8	sle_ssaps_set_info()	设置 MTU 为 1500 字节
9	sle_speed_connect_param_init()	设置默认连接参数 (扫描间隔/窗口、连接间隔)
10	sle_set_local_addr_init()	固定本地 MAC 地址为 11:22:33:44:55:66

3.3 UUID 设计与 Property 注册

SLE 使用 128 位 UUID 基地址 + 2 位偏移的方式定义 UUID。sle_uuid_base 是全局基地址数组，sle_uuid_setu2() 在 base 的第 14~15 字节填入具体的 2 字节 UUID 值：

```
static uint8_t sle_uuid_base[] = {
    0x37, 0xBE, 0xA8, 0x80, 0xFC, 0x70, 0x11, 0xEA,
    0xB7, 0x20, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00
};

/* 将2字节 u2 写入 base[14..15] (小端序) */
static void sle_uuid_setu2(uint16_t u2, sle_uuid_t *out) {
    sle_uuid_set_base(out);
    out->len = 2;
    encode2byte_little(&out->uuid[14], u2);
}
```

Property 的 operate_indication 字段决定其权限：SSAP_OPERATE_INDICATION_BIT_READ | SSAP_OPERATE_INDICATION_BIT_NOTIFY 表示该特征值可被读取，也可以由 Server 主动 Notify 推送给 Client。

3.4 数据 Notify 发送

数据发送通过 ssaps_notify_indicate() 实现。Server 在 ssaps_read_request_cbk 被触发时（Client 读属性时），创建 "RadarTask" 内核线程，由该线程循环发送数据帧：

```
errcode_t sle_uuid_server_send_report_by_handle_id(
    uint8_t *data, uint16_t len, uint16_t connect_id) {
    ssaps_ntf_ind_t param = {0};
    param.handle = g_property_handle; // Property 句柄
    param.type = SSAP_PROPERTY_TYPE_VALUE;
    param.value = data;
    param.value_len = len;
    ssaps_notify_indicate(g_server_id, connect_id, &param);
    return ERRCODE_SLE_SUCCESS;
}
```

关键：param.handle 必须是 g_property_handle（ssaps_add_property_sync 返回值），不能是 Descriptor 的 handle，否则 Client 端 Notify 回调不触发。

3.5 广播参数配置

Server 使用以下参数发起广播（Announce），Client 扫描到该广播后发起连接：

参数	值	含义
announce_mode	CONNECTABLE_SCANA BLE	可连接、可扫描
announce_interval_min/ max	0xC8 (25ms×8=200ms)	广播间隔约 200ms

参数	值	含义
conn_interval_min/max	0xA (10×125μs=1.25ms)	连接间隔 1.25ms（高速）
conn_supervision_timeout	0x1F4 (5000ms)	连接监督超时 5 秒
announce_tx_power	20 dBm	最大发射功率
own_addr	11:22:33:44:55:66	固定 MAC，便于 Client 精准识别

第四章 WS63 Board 2/3 —— SLE Client 执行节点

4.1 Board 2 的职责（座椅电机节点）

- 开机后扫描 SLE 广播，找到 MAC 为 11:22:33:44:55:66 的 Board 1；
- 停止扫描，向 Board 1 发起连接请求；
- 连接成功后，发现 Board 1 的 SSAP 服务和 Property，订阅 Notify；
- 接收 Board 1 发来的 Notify 数据（含电机步进指令 F5/R5）；
- 解析指令并驱动座椅步进电机（前进/后退各 5 步）。

4.1b Board 3 的职责（氛围灯节点）

- 开机后扫描 SLE 广播，找到 MAC 为 11:22:33:44:55:66 的 Board 1；
- 停止扫描，向 Board 1 发起连接请求（Board 1 同时维护两个 Client 连接）；
- 连接成功后，发现 Board 1 的 SSAP 服务和 Property，订阅 Notify；
- 接收 Board 1 发来的 Notify 数据（含氛围灯指令 L1/L0）；
- 解析指令并切换氛围灯 LED 的开/关状态。

4.2 Client 连接流程

Client 的连接过程通过一系列回调函数串联驱动：

回调函数	触发时机	主要操作
sle_sample_sle_enable_cbk	SLE 协议栈使能完成	调用 sle_start_scan() 开始扫描
sle_sample_seek_result_info_cbk	收到一条广播结果	比较 MAC，匹配则保存地址并调用 sle_stop_seek()
sle_sample_seek_disable_cbk	扫描停止	调用 sle_connect_remote_device() 发起连接
sle_connect_state_changed_cbk	连接状态改变	连接成功后保存 conn_id，发起服务发现
sle_speed_notification_cb	收到 Server 的 Notify	解析数据，计算吞吐率（本项目中扩展为解析指令）

4.3 扫描精准识别目标 Board 1

```
void sle_sample_seek_result_info_cbk(
sle_seek_result_info_t *seek_result_data) {
uint8_t mac[SLE_ADDR_LEN] = {0x11,0x22,0x33,0x44,0x55,0x66};
if (memcmp(seek_result_data->addr.addr, mac, SLE_ADDR_LEN) == 0) {
memcpy_s(&g_remote_addr, sizeof(sle_addr_t),
&seek_result_data->addr, sizeof(sle_addr_t));
sle_stop_seek(); // 停止扫描，准备连接
}
}
```

Client 扫描到每个广播包时触发 seek_result_info_cbk，逐一比较 MAC 地址。当 MAC 完全匹配 Board 1 的固定地址后，调用 sle_stop_seek()，其完成回调 seek_disable_cbk 中再发起连接。这种模式确保 Client 只连接指定 Server，适合多节点环境下的精准配对。

第五章 UART 传感器帧协议详解

5.1 帧格式 (11 字节)

Board 1 与 GEC6818 之间的通信使用自定义二进制帧，固定 11 字节。UART 参数为 115200 bps, 8 数据位，无奇偶校验，1 停止位 (8N1) 。

字节偏移	字段名	类型	说明
[0]	帧头 1	uint8	固定值 0xAA
[1]	帧头 2	uint8	固定值 0x55
[2]	温度高字节	int16 高 8 位	温度 × 10，有符号 (如 256 = 25.6℃)
[3]	温度低字节	int16 低 8 位	
[4]	湿度高字节	uint16 高 8 位	湿度 × 10 (如 653 = 65.3%)
[5]	湿度低字节	uint16 低 8 位	
[6]	MQ-135 高字节	uint16 高 8 位	MQ-135 电压 (mV)
[7]	MQ-135 低字节	uint16 低 8 位	
[8]	MQ-135 数字	uint8	0=正常，1=超标
[9]	XOR 校验	uint8	[2]⊕[3]⊕[4]⊕[5]⊕[6]⊕[7]⊕[8]
[10]	帧尾	uint8	固定值 0x0D (回车符)

5.2 接收端状态机解析 (GEC6818 侧)

GEC6818 的 show_sensor_thread 使用三态状态机逐字节解析帧，避免了固定缓冲区偏移导致的丢帧问题：

```

int state = 0, idx = 0;
unsigned char byte;
unsigned char frame[SENSOR_FRAME_LEN]; // 11 字节

while (1) {
    read(fd, &byte, 1); // 每次读 1 字节
    switch (state) {
        case 0: // 等待帧头1
            if (byte == 0xAA) { frame[0]=byte; state=1; }
            break;
        case 1: // 等待帧头2
            if (byte == 0x55) { frame[1]=byte; idx=2; state=2; }
            else { state=0; } // 不匹配, 回到初态
            break;
        case 2: // 接收剩余 9 字节
            frame[idx++] = byte;
            if (idx < SENSOR_FRAME_LEN) break;
            state = 0; // 帧完整, 重置状态机
            // 1. 校验帧尾
            if (frame[10] != 0x0D) { break; }
            // 2. XOR 校验
            uint8_t xc = frame[2]^frame[3]^frame[4]^
            frame[5]^frame[6]^frame[7]^frame[8];
            if (xc != frame[9]) { break; }
            // 3. 解析有效数据
            int16_t tr = (int16_t)(((uint16_t)frame[2]<<8)|frame[3]);
            uint16_t hr = ((uint16_t)frame[4]<<8)|frame[5];
            uint16_t mv = ((uint16_t)frame[6]<<8)|frame[7];
            uint8_t dig = frame[8];
            g_sensor_temp = tr / 10.0; // 恢复真实温度
            g_sensor_humi = hr / 10.0; // 恢复真实湿度
            break;
        }
    }
}

```

状态机的优势：无论串口噪声导致哪个字节错位，只要找不到合法的 0xAA + 0x55 双字节头，状态机就会持续等待，不会把噪声误判为有效帧。这比固定偏移读法健壮得多。

5.3 串口初始化参数 (termios)

```
static void serial_init(int fd) {
    struct termios t;
    bzero(&t, sizeof(t));
    cfmakeraw(&t); // 原始模式, 不做行缓冲
    cfsetispeed(&t, B115200); // 输入波特率
    cfsetospeed(&t, B115200); // 输出波特率
    t.c_cflag |= CLOCAL | CREAD; // 忽略调制解调器状态线, 启用接收
    t.c_cflag &= ~CSIZE;
    t.c_cflag |= CS8; // 8 位数据
    t.c_cflag &= ~PARENB; // 无奇偶校验
    t.c_cflag &= ~CSTOPB; // 1 位停止位
    t.c_cc[VMIN] = 0; // 最少读 0 字节 (非阻塞)
    t.c_cc[VTIME] = 20; // 2 秒超时 (单位: 1/10 秒)
    tcflush(fd, TCIOFLUSH); // 清空收发缓冲区
    tcsetattr(fd, TCSANOW, &t);
}
```

第六章 GEC6818 上位机 —— LCD Framebuffer 与显示

6.1 Linux Framebuffer 原理

GEC6818 运行 Linux，LCD 驱动将显存抽象为字符设备 /dev/fb0。应用程序通过 mmap 将显存映射到进程虚拟地址空间，之后对该内存的写入会立即反映到屏幕上，无需系统调用，效率极高。

本项目屏幕参数：800×480，32 位色深（BGRA 格式，4 字节/像素）。每个像素在 lcd_map 中的偏移为 $(y * 800 + x) * 4$ ，字节顺序为 B（蓝）、G（绿）、R（红）、A（保留）。

```
int lcd_fb_init() {  
    lcd_fd = open("/dev/fb0", O_RDWR);  
    lcd_map = mmap(NULL,  
        800 * 480 * 4, // 总字节数  
        PROT_READ | PROT_WRITE,  
        MAP_SHARED, // 共享映射，写入即显示  
        lcd_fd, 0);  
    return 0;  
}
```

6.2 BMP 图片加载与显示

CID 界面背景图以 BMP 格式存储。BMP 文件头（54 字节）包含宽度、高度、色深等信息。BMP 像素按行从底部到顶部存储（Y 轴翻转），因此读取时需要反转 y 坐标：

```

void lcd_show_bmp(unsigned char *lcd, const char *filename) {
    int bmp_fd = open(filename, O_RDONLY);
    char header[54];
    read(bmp_fd, header, 54);
    int bmp_w = *(int*)&header[18]; // 宽度
    int bmp_h = *(int*)&header[22]; // 高度
    int bmp_pixel_size = *(short*)&header[28] / 8; // 字节/像素

    unsigned char *bmp_data = malloc(bmp_w * bmp_h * bmp_pixel_size);
    read(bmp_fd, bmp_data, bmp_w * bmp_h * bmp_pixel_size);

    int offset_x = (800 - bmp_w) / 2; // 居中显示
    int offset_y = (480 - bmp_h) / 2;

    for (int y = 0; y < bmp_h; y++) {
        for (int x = 0; x < bmp_w; x++) {
            // BMP y轴翻转: bmp_h-1-y
            int bmp_idx = ((bmp_h-1-y) * bmp_w + x) * bmp_pixel_size;
            int lcd_idx = ((y+offset_y) * 800 + (x+offset_x)) * 4;
            lcd[lcd_idx+0] = bmp_data[bmp_idx+0]; // B
            lcd[lcd_idx+1] = bmp_data[bmp_idx+1]; // G
            lcd[lcd_idx+2] = bmp_data[bmp_idx+2]; // R
            lcd[lcd_idx+3] = 0x00;
        }
    }
    // 保存背景副本，用于文字区域的背景恢复
    memcpy(g_bg_image, lcd, 800*480*4);
}

```

`g_bg_image` 是 BMP 背景的内存副本。当分区上需要显示文字时，先从 `g_bg_image` 恢复该分区的背景像素，再在其上叠加透明文字，从而避免文字背景色块遮挡 BMP 图案。

6.3 文字渲染 (TrueType 字库)

本项目使用 FreeType 风格的自定义 TrueType 渲染库 (`src/font.c + src/truetype.c`)，加载 `/usr/share/fonts/simfang.ttf` (仿宋字体)，支持中文渲染。

`font_show()` 函数的参数说明：

参数	含义
<code>text</code>	要显示的文本 (支持中文 UTF-8)
<code>pixelSize</code>	字体像素大小 (最大 72)
<code>outFrameWidth/Height</code>	文字输出框的宽/高 (像素)
<code>outFrameColor</code>	输出框背景色；填 0 时不填充背景色，改为从 <code>g_bg_image</code> 恢复

参数	含义
fontPosX/Y	文字在输出框内的起始坐标
fontColor	文字颜色（ARGB 格式，如 0x0040FF80 = 绿色）
frameToLcdPosX/Y	输出框左上角在 LCD 上的坐标

showbitmap() 将渲染好的字体位图叠加到 lcd_map：RGB 全为 0 的像素视为透明背景，跳过不写，保留 BMP 内容可见。

第七章 触摸屏处理与 8 分区交互设计

7.1 触摸事件读取原理

GEC6818 的触摸屏驱动注册为 Linux 输入子系统设备 /dev/input/event0。应用程序读取 struct input_event 结构体 (type/code/value 三元组) 解析触摸动作：

type	code	value	含义
EV_ABS	ABS_X	0~1023	触摸点 X 坐标 (原始值)
EV_ABS	ABS_Y	0~613	触摸点 Y 坐标 (原始值)
EV_KEY	BTN_TOUCH	1	手指按下 (Pressed)
EV_KEY	BTN_TOUCH	0	手指抬起 (Released)

原始坐标需要映射到屏幕坐标：

$$\text{screen_x} = \text{raw_x} \times 800 / 1024$$

$$\text{screen_y} = \text{raw_y} \times 480 / 614$$

主循环使用 select() 以 50ms 超时轮询触摸设备（非阻塞），这样即使没有触摸事件，主线程仍然可以处理座椅持续按压逻辑（每 60ms 重发步进指令）。

7.2 8 个触摸分区定义

屏幕被划分为 8 个矩形热区（对应 CID2.bmp 背景图上的功能区）：

编号	宏名称	坐标范围 (x,y,w,h)	功能
0	ZONE_TIME	(0,0,400,160)	显示当前系统时间
1	ZONE_WEATHER	(400,0,399,160)	显示合肥实时天气
2	ZONE_TEMP	(0,160,268,160)	显示当前温度 (SHT30)
3	ZONE_HUMI	(268,160,267,160)	显示当前湿度 (SHT30)
4	ZONE_AIR	(535,160,264,160)	显示空气质量 (MQ-135)
5	ZONE_SEAT_UP	(0,320,268,160)	座椅电机上调 (持续按压)
6	ZONE_SEAT_DN	(268,320,267,160)	座椅电机下调 (持续按压)

编号	宏名称	坐标范围 (x,y,w,h)	功能
7	ZONE_LIGHT	(535,320,264,159)	氛围灯开关切换

7.3 触摸事件处理逻辑

按下 (BTN_TOUCH=1) 和抬起 (BTN_TOUCH=0) 分别处理：

分区	按下时	抬起时	额外逻辑
ZONE_TIME /WEATHER/ TEMP/HUM I/AIR	记录 pressed_zone	调用 show_zone_data(zone) 显示数据; 设置 5 秒显示倒计时	zone_restore_thread 每 100ms 检查倒计时, 到期恢复 BMP 背景
ZONE_SEAT _UP	显示"上调中..."文字 ; 发送 "F5\r\n" 给 Board 1	恢复 BMP 背景	持续按压期间每 60ms 重发 "F5\r\n" (步进电机连续控制)
ZONE_SEAT _DN	显示"下调中..."文字 ; 发送 "R5\r\n" 给 Board 1	恢复 BMP 背景	持续按压期间每 60ms 重发 "R5\r\n"
ZONE_LIGH T	(不处理)	切换 g_light_on 状态; 发送 "L1\r\n" 或 "L0\r\n"; 更新文字显示	300ms 防抖: 两次点击间隔 < 300ms 忽略

7.4 座椅指令通过 UART 发送给 Board 1

```
/* main.c — send_seat_cmd() */
static void send_seat_cmd(const char *cmd) {
pthread_mutex_lock(&g_board1_write_mutex);
if (g_board1_fd >= 0) {
write(g_board1_fd, cmd, strlen(cmd));
printf("[座椅] 发送指令: %s", cmd);
}
pthread_mutex_unlock(&g_board1_write_mutex);
}

/* 指令字符串含义 */
// "F5\r\n" —— Forward 5步（座椅向前/向上）
// "R5\r\n" —— Reverse 5步（座椅向后/向下）
// "L1\r\n" —— Light On（氛围灯开）
// "L0\r\n" —— Light Off（氛围灯关）
```

g_board1_fd 由 show_sensor_thread 在打开 UART 串口时赋值，主线程通过互斥锁 g_board1_write_mutex 保护并发写入。Board 1 收到此指令后，按前缀（F/R → 电机，L → 灯光）分别通过 SLE Notify 转发给 Board 2（电机节点）或 Board 3（灯光节点）。

第八章 HTTP 天气获取 (ShowAPI)

8.1 实现原理

本项目不依赖任何第三方 HTTP 库，直接使用 POSIX Socket TCP 发送 HTTP GET 请求，手工解析响应头和 JSON body。show_weather_thread 线程每 10 分钟调用一次 fetch_weather()，将结果写入全局变量 g_weather_str。

8.2 天气获取流程

1. 调用 gethostbyname() 解析 ali-weather.showapi.com 的 IP 地址；
2. 创建 TCP Socket, connect() 到 80 端口 (HTTP) , 设置 10 秒收发超时；
3. 构建 HTTP GET 请求, URL 中城市参数为合肥 (%E5%90%88%E8%82%A5) , Authorization 头携带 APPCODE；
4. write() 发送请求, 逐字节读取直到出现 \r\n\r\n (HTTP 头结束) ；
5. 检查响应码是否含 "200", 否则中止；
6. 循环 read() 读取 JSON body (最多 16 KB) ；
7. 用 json_get_str() 从 JSON 中提取 "weather" 和 "wind_direction" 字段；
8. 拼接成 "合肥 晴 东风" 格式字符串, 加互斥锁写入 g_weather_str。

8.3 JSON 解析函数

```
/* 从 JSON 字符串中提取 "key": "value" */
static int json_get_str(const char *json, const char *key,
char *dst, int dstsz) {
char search[80];
snprintf(search, sizeof(search), "\"%s\":\\\"", key);
const char *p = strstr(json, search);
if (!p) return 0;
p += strlen(search); // 跳过 "key":
int i = 0;
while (*p && *p != '"' && i < dstsz-1)
dst[i++] = *p++; // 复制 value 直到结束引号
dst[i] = '\\0';
return (i > 0);
}
```

ShowAPI 接口返回的 JSON 字段名因 API

版本不同而异 (weather/wea、wind_direction/windDirection/wind) , 代码用多次 fallback 依次尝试, 增强了对 API 变更的鲁棒性。

第九章 OneNET MQTT 数据上传

9.1 MQTT 3.1.1 协议基础

MQTT (Message Queuing Telemetry Transport) 是一种轻量级发布/订阅协议，基于 TCP，适合 IoT 设备。本项目不使用第三方 MQTT 库，而是手工构建 MQTT 数据包。连接到 OneNET 平台 (mqttps.heclouds.com:1883) 需要以下认证信息：

MQTT 字段	值	说明
Client ID	device_1	设备唯一标识
Username	QmP8P8t72k	OneNET 产品 ID
Password	Token 字符串 (MD5 签名)	含过期时间和签名的鉴权 Token
Keep-alive	60 秒	心跳间隔，超时连接断开
Clean Session	1 (0xC2 标志位)	每次连接不保留旧会话

9.2 MQTT 数据包手工构建

CONNECT 包结构 (固定头 + 可变头 + Payload)：

```
/* 构建 MQTT CONNECT 包 */
static int mqtt_build_connect(uint8_t *buf) {
    int payload_len = 2+len(ClientID) + 2+len(Username) + 2+len>Password);
    int remaining = 10 + payload_len; // 10 = 可变头固定长度
    int pos = 0;
    buf[pos++] = 0x10; // 控制包类型 CONNECT
    pos += mqtt_encode_len(buf+pos, remaining); // 剩余长度 (可变长编码)
    // 协议名 "MQTT" (4字节) + 版本 0x04 + 标志 0xC2 + Keep-alive 0x003C
    buf[pos++] = 0x00; buf[pos++] = 0x04;
    buf[pos++] = 'M'; buf[pos++] = 'Q'; buf[pos++] = 'T'; buf[pos++] = 'T';
    buf[pos++] = 0x04; // MQTT 3.1.1
    buf[pos++] = 0xC2; // Username+Password+CleanSession
    buf[pos++] = 0x00; buf[pos++] = 0x3C; // Keep-alive 60s
    pos += mqtt_write_str(buf+pos, ClientID);
    pos += mqtt_write_str(buf+pos, Username);
    pos += mqtt_write_str(buf+pos, Password);
    return pos;
}
```

PUBLISH 包 (QoS 0, 无需 ACK)：

```
static int mqtt_build_publish(uint8_t *buf,
const char *topic, const char *payload) {
int tlen = strlen(topic), plen = strlen(payload);
int remaining = 2 + tlen + plen;
int pos = 0;
buf[pos++] = 0x30; // PUBLISH, QoS=0, Retain=0
pos += mqtt_encode_len(buf+pos, remaining);
buf[pos++] = (tlen>>8)&0xFF; buf[pos++] = tlen&0xFF;
memcpy(buf+pos, topic, tlen); pos += tlen;
memcpy(buf+pos, payload, plen); pos += plen;
return pos;
}
```

9.3 上报的 JSON Payload 格式

OneNET 物模型数据上报格式 (Topic: \$sys/{产品ID}/{设备ID}/thing/property/post) :

```
{
  "id": "1",
  "version": "1.0",
  "params": {
    "temp_value": { "value": 25.6 }, // 温度 °C
    "humidity_value": { "value": 65.3 }, // 湿度 %
    "historical_record": { "value": 1200 } // MQ-135 电压 mV
  }
}
```

9.4 MQTT 线程运行机制

mqtt_upload_thread 采用断线重连机制，主循环结构如下：

```
while (1) {
  if (fd < 0) {
    fd = tcp_connect(ONENET_BROKER, ONENET_PORT);
    // 发送 CONNECT, 等待 CONNACK (20 02 00 00)
    // 失败则 30 秒后重试
  }
  // 读取传感器全局变量 (加锁)
  // 构建 JSON Payload
  // 发送 PUBLISH
  sleep(55); // 等待 55 秒
  mqtt_build_pingreq(buf); // 发送 PINGREQ 保活
  read(fd, pong, 2); // 读取 PINGRESP (可忽略)
  sleep(5); // 再等 5 秒, 总间隔约 60 秒上传一次
}
```

Keep-alive 设置为 60 秒，MQTT 规定超过 $1.5 \times \text{Keep-alive}$ (90秒) 无活动则断连。线程每 55 秒发一次 PINGREQ 心跳包，确保连接不超时断开。

第十章 多线程架构与并发安全

10.1 线程概览

线程	入口函数	主要任务	唤醒周期
主线程	main()	触摸事件处理，发送座椅/灯光指令	select() 50ms 轮询
传感器线程	show_sensor_thread()	UART 接收传感器帧，更新全局变量	阻塞 read(), 2s 超时
天气线程	show_weather_thread()	HTTP 拉取天气，更新 g_weather_str	sleep(600), 10分钟
MQTT 上传线程	mqtt_upload_thread()	MQTT 上传传感器数据到 OneNET	sleep(60), 约60秒
分区恢复线程	zone_restore_thread()	检查信息区 5 秒倒计时，到期恢复背景	usleep(100000), 100ms

10.2 互斥锁保护的全局资源

互斥锁	保护的资源	使用线程
lcd_mutex	lcd_map (Framebuffer)	主线程 + 传感器线程 (font_show 内)
g_sensor_mutex	g_sensor_temp/humi/mq_mv/ mq_dig	传感器线程 (写) + MQTT线程 (读) + 主线程 (读)
g_board1_write_mutex	g_board1_fd (UART 写入)	主线程 (写指令)
g_weather_mutex	g_weather_str	天气线程 (写) + 主线程 (读, ZONE_WEATHER 触发)
g_zone_mutex	g_zone_expire[5] (倒计时数组)	主线程 (写) + 分区恢复线程 (读)

10.3 线程启动代码

```
// main() 中启动四个子线程
pthread_t tid_sensor, tid_weather, tid_mqtt, tid_zone;
pthread_create(&tid_sensor, NULL, show_sensor_thread, NULL);
pthread_create(&tid_weather, NULL, show_weather_thread, NULL);
pthread_create(&tid_mqtt, NULL, mqtt_upload_thread, NULL);
pthread_create(&tid_zone, NULL, zone_restore_thread, NULL);
```

注意：g_board1_fd 初始值为 -1，由 show_sensor_thread 在成功打开 /dev/ttySAC1 后赋值。主线程的 send_seat_cmd() 在发送前检查 g_board1_fd >= 0，若串口尚未就绪则跳过，不会因为过早调用而崩溃。

第十一章 工程编译与运行

11.1 GEC6818 上位机编译

使用交叉编译工具链 arm-linux-gcc, Makefile 如下:

```
CC = arm-linux-gcc
TARGET = sle_cid
CFLAGS = -I inc/
LDFLAGS = -lpthread -lm

SRCS = main.c \
src/cid_display.c \
src/sle_serial.c \
src/onenet_mqtt.c \
src/font.c \
src/truetype.c

$(TARGET): $(SRCS)
$(CC) $(CFLAGS) -o $(TARGET) $(SRCS) $(LDFLAGS)

clean:
rm -f $(TARGET)
```

编译步骤 (在 Ubuntu 交叉编译环境中) :

```
# 1. 进入项目目录
cd /path/to/sle_CID

# 2. 执行编译
make

# 3. 通过 NFS 或 SCP 传输到 GEC6818
scp sle_cid root@:/home/
scp -r images/ root@:/home/

# 4. 在 GEC6818 上运行
chmod +x sle_cid
./sle_cid /home/images/
```

11.2 WS63 固件编译与烧录

WS63 使用华为 HiSpark Studio 或命令行编译:

```
# SDK 根目录下执行
./build.py --product HiHope_NearLink_DK_WS63E_V03 --target sle_speed_server

# 烧录: 使用 HiSpark Studio 串口烧录工具
# 波特率: 115200, UART0 用于烧录和调试
```

Board 1 烧录 sle_speed_server 固件（含 SHT30/MQ-135 传感器采集 + 双 Client 路由逻辑）；Board 2 烧录 sle_speed_client 固件（含座椅步进电机控制逻辑）；Board 3 烧录 sle_speed_client2 固件（含氛围灯 LED 控制逻辑）。编译时注意 CMakeLists.txt 中的 app_component 配置指向正确的源文件目录。

11.3 运行时日志示例

```
$ ./sle_cid /home/images/  
Framebuffer initialized (800x480).  
系统启动成功，开始 CID 显示...  
[MQTT] OneNET 上传线程启动  
[串口] 已打开 /dev/ttySAC1，等待 Board 1 传感器数据...  
[天气] 连接 ali-weather.showapi.com...  
[传感器] 温度:25.6°C 湿度:63.5% MQ-135:1230mV(正常)  
[MQTT] 已连接到 OneNET  
[MQTT] 已上传: 温度=25.6°C 湿度=63.5% 空气质量=1230mV  
[天气] 更新: 合肥 晴 东风  
[触摸] 按下 x=150 y=80 zone=0  
[触摸] 抬起 x=150 y=80 zone=0  
[触摸] 按下 x=100 y=350 zone=5  
[座椅] 发送指令: F5  
[触摸] 抬起 x=100 y=350 zone=5
```

第十二章 工程目录结构与文件职责

12.1 GEC6818 上位机目录

文件/目录	职责
main.c	程序入口；触摸主循环；座椅/灯光指令发送
inc/sle_CID.h	全局宏定义（分区坐标、UART设备、MQTT配置）；全局变量 extern 声明；函数声明
src/cid_display.c	LCD 初始化 (lcd_fb_init) ； BMP 显示 (lcd_show_bmp) ； 文字渲染 (font_show) ； 分区背景恢复 (restore_zone_bg) ； 天气线程；分区恢复线程 (zone_restore_thread)
src/sle_serial.c	UART 串口初始化；传感器帧接收线程 (show_sensor_thread) ； 全局传感器变量定义
src/onenet_mqtt.c	TCP Socket 封装 (tcp_connect) ； HTTP 天气获取 (fetch_weather) ； MQTT 包构建 (CONNECT/PUBLISH/PINGREQ) ； MQTT 上传线程
src/font.c	字库加载、字符渲染、位图操作
src/truetype.c	TrueType 字体解析 (FreeType 风格)
inc/font.h	字库类型和函数声明
inc/lcd.h	LCD 基础接口声明
inc/bitmap.h	位图结构体和操作声明
inc/truetype.h	TrueType 渲染接口声明
images/CID1.bmp	界面背景图 1（无分区线）
images/CID2.bmp	界面背景图 2（带分区线，开发调试用）
Makefile	交叉编译规则

12.2 WS63 SDK SLE 相关文件（Server 侧）

文件	职责
sle_speed_server.c	Server 服务注册；回调处理；向 Board 2/3 的数据 Notify 路由（按 conn_id 区分）；连接参数管理
sle_speed_server_adv.c	广播参数设置；广播数据构建；广播使能
sle_speed_server.h	UUID 宏定义；Server 函数声明
sle_speed_server_adv.h	广播相关函数声明

文件	职责
sle_speed_client.c	Board 2 Client 逻辑；扫描→连接→订阅 Notify；解析 F5/R5 指令驱动步进电机
sle_speed_client2.c	Board 3 Client 逻辑；与 Board 2 相同的连接流程；解析 L1/L0 指令控制氛围灯 LED

第十三章 常见问题与排查

13.1 SLE 相关

问题现象	可能原因	排查方法
Client 扫描不到 Board 1	Board 1 未启动广播; MAC 地址不匹配	检查 Board 1 日志中 "sle_uuid_server_adv_init out" 是否出现; 确认 Client 中的目标 MAC 与 Server 的 sle_set_local_addr_init() 一致
Board 2/3 收不到 Notify 数据	1. Client 发起读请求后 Server 才开始发送; 2. Notify 用了 Descriptor handle 而非 Property handle; 3. Board 1 Notify 时 conn_id 填错 (混淆了 Board 2 和 Board 3 的 conn_id)	确保 Client ssapc_find_service 后调用一次读操作触发 Server 发送; 确认 ssaps_notify_indicate 的 param.handle = g_property_handle; 检查 Board 1 是否正确记录了两个 Client 的 conn_id 并按指令类型分发
CONNACK 返回非 0x00 (连接失败)	OneNET Token 过期或用户名/密码错误	检查 sle_CID.h 中 ONENET_PASSWORD 的 Token 是否已过期; 登录 OneNET 控制台重新生成 Token
LCD 显示黑屏	/dev/fb0 权限不足; mmap 失败	chmod 666 /dev/fb0; 检查内核是否加载 framebuffer 驱动
文字乱码 (方块字)	字库文件不存在或路径错误	确认 /usr/share/fonts/simfang.ttf 存在; 检查 fontLoad 返回值
触摸坐标偏移	X/Y 轴原始值范围与代码预设不符	修改 SCREEN_W/1024 和 SCREEN_H/614 的分母为实际触摸屏分辨率
UART 接收到乱码/校验失败	Board 1 UART TX 与 GEC6818 UART1 RX 接线错误; 波特率不一致	检查接线 (TX→RX, GND 共地); 确认 Board 1 和 GEC6818 两侧均配置为 115200 8N1

13.2 编译常见错误

错误信息	原因	解决方法
arm-linux-gcc: command not found	交叉编译工具链未安装或未加入 PATH	export PATH=\$PATH:/opt/arm-linux-gcc/bin
-lpthread 链接报错	未加链接选项	Makefile LDFLAGS 中确保有 -lpthread -lm

错误信息	原因	解决方法
implicit declaration of 'fontLoad'	未包含头文件	#include "font.h" 已在 sle_CID.h 中, 确认 CFLAGS 有 -I inc/

第十四章 知识点总结

14.1 本项目覆盖的核心知识点

知识领域	具体知识点
无线通信	星闪 SLE 协议栈；SSAP 服务/特征值/描述符；Server/Client 广播与连接；1 Server 对 2 Client Notify 路由；PHY 速率配置
Linux 系统编程	Framebuffer /dev/fb0 mmap；Input 子系统 /dev/input/event0；termios 串口配置；POSIX pthread 多线程；互斥锁 pthread_mutex
网络编程	TCP Socket；HTTP 手工请求与响应解析；MQTT 3.1.1 包结构手工构建；CONNACK/PUBLISH/PINGREQ
嵌入式 C	BMP 文件格式解析（文件头+像素数据）；TrueType 字体渲染；状态机协议解析；XOR 校验
物联网	OneNET 物模型数据上报；MQTT 鉴权 Token；ShowAPI 天气接口
工程实践	交叉编译（arm-linux-gcc）；Makefile 构建；多线程并发安全设计；非阻塞 I/O（select+超时）

14.2 系统架构总结图示

GEC6818 上位机 (Linux ARM)

主线程（触摸） | show_sensor_thread (UART接收) | mqtt_upload_thread (MQTT上传) |
show_weather_thread (HTTP天气) | zone_restore_thread (分区恢复)

LCD /dev/fb0 mmap ↔ 触摸 /dev/input/event0

GEC6818	UART 双向 115200 8N1	Board 1 WS63E (SLE Server)	SLE Notify (F5/R5)	Board 2 WS63E (电机节点)
MQTT TCP		SHT30 I2C MQ-135 ADC	SLE Notify (L1/L0)	Board 3 WS63E (灯光节点)
OneNET 云平台		传感器+路由节点		步进电机 / 氛围灯

至此，本项目的所有关键模块已全面讲解完毕。建议按以下顺序学习和复现：

- 步骤 1：理解星闪 SLE 协议基础（第二章），熟悉 SSAP API；
- 步骤 2：烧录 Board 1 Server 固件，观察广播和传感器数据输出（第三章）；

- 步骤 3: 烧录 Board 2 Client 固件, 确认 SLE 连接建立, Notify 数据正常接收 (第四章);
- 步骤 4: 理解 UART 帧格式, 在 PC 上用串口工具模拟验证状态机解析 (第五章);
- 步骤 5: 在 GEC6818 上运行上位机程序, 验证 LCD 显示和触摸功能 (第六~七章);
- 步骤 6: 配置 OneNET 设备, 验证 MQTT 数据上报 (第九章);
- 步骤 7: 结合项目资料模板, 填写完整的技术文档。